

Transformer Computational Theory

formal languages, circuit complexity, Turing completeness, chain-of-thought computation

이번 주의 질문은 “Transformer가 어떤 연속함수를 근사할 수 있는가?”에서 한 단계 더 나아가간다. 입력 길이가 커질 때, 제한된 **depth**와 **precision**을 가진 **Transformer**가 실제로 어떤 계산 문제를 풀 수 있는가? 이 질문에는 서로 다른 답이 공존한다. 이상화된 arbitrary-precision hard-attention Transformer는 Turing complete일 수 있지만, 현실적인 log-precision의 고정-depth Transformer 한 번의 forward pass는 uniform TC^0 로 upper bound된다. 그리고 autoregressive intermediate generation을 허용하면 생성 step 수 자체가 새로운 순차 계산 자원이 되어, 선형 길이의 scratchpad는 모든 regular language를, 다항 길이의 scratchpad는 적절한 가정 아래 정확히 P 수준의 계산을 지원한다. 이번 노트의 목적은 이 결과들이 서로 모순이 아니라 서로 다른 **resource model**에 대한 정리라는 점을 수식으로 분리하는 것이다.

Primary readings

Jorge Pérez, Pablo Barceló, Javier Marinkovic, **Attention is Turing Complete**, JMLR 2021.

William Merrill, Ashish Sabharwal, **The Parallelism Tradeoff: Limitations of Log-Precision Transformers**, TACL 2023.

William Merrill, Ashish Sabharwal, **The Expressive Power of Transformers with Chain of Thought**, ICLR 2024.

Michael Hahn, **Theoretical Limitations of Self-Attention in Neural Sequence Models**, TACL 2020.

Contents

1	먼저 맞춰 둘 기호와 전체 그림	3
1.1	이번 주에 섞이면 안 되는 네 개의 축	3
1.2	논문별 기호 대응표	3
1.3	결론을 먼저 보는 지도	3
2	Formal language와 circuit complexity의 최소 배경	5
2.1	Language recognition을 계산 문제로 보기	5
2.2	Finite automaton, regular language, 그리고 sequentiality	5
2.3	Circuit family: input length마다 회로 하나	6
2.4	AC^0 , TC^0 , NC^1	6
2.5	왜 TC^0 가 생각보다 약하지 않은가	6
3	고정-depth self-attention의 초기 한계	8
3.1	Hahn의 질문: recurrence 없이 hierarchy를 처리할 수 있는가?	8
3.2	Hard attention을 회로로 보면 더 깔끔해진다	8
3.3	“attention이 global이니까 sequential problem도 쉽다”가 왜 틀릴 수 있는가	9
4	Log-precision Transformer와 TC^0 upper bound	10
4.1	왜 precision을 명시해야 하는가?	10
4.2	Main theorem	10
4.3	왜 circuit depth가 constant인가? — proof skeleton	10
4.4	Self-attention의 합이 왜 “순차 합산”을 요구하지 않는가?	11
4.5	Upper bound의 실제 의미	11
5	그런데 왜 Transformer가 Turing complete인가?	13
5.1	Turing completeness의 정의부터 분리한다	13
5.2	Pérez–Barceló–Marinkovic의 main theorem	13
5.3	TM simulation의 상태를 무엇으로 저장하는가?	13
5.4	가장 최근 write를 찾으려면 tape를 읽을 수 있다	14
5.5	Positional encoding이 왜 핵심인가?	14
5.6	Fixed precision이면 왜 같은 증명이 무너지는가?	14
6	모순 해소: one-pass TC^0와 Turing completeness	16
6.1	두 theorem의 assumptions를 같은 표에 놓기	16
6.2	계산 resource를 식으로 분해하면 더 명확하다	16
6.3	Precision도 독립적인 resource다	16
7	Chain of Thought를 계산 resource로 보기	18
7.1	Intermediate token 하나는 forward pass 하나다	18
7.2	Merrill–Sabharwal의 central sandwich	18
7.3	왼쪽 inclusion: TM step 하나를 token 하나로	18
7.4	Layer-norm hash를 한 줄씩 유도	19
7.5	Regular language는 왜 linear CoT면 충분한가?	20
7.6	Polynomial CoT가 왜 P까지 가는가?	20

8 CoT 길이에 따른 계산력의 계층	21
8.1 $O(1)$: intermediate generation이 없는 경우	21
8.2 $O(\log n)$: 조금의 scratchpad는 얼마나 도움이 되는가?	21
8.3 $\Theta(n)$: 모든 regular language	21
8.4 $\text{poly}(n)$: 정확히 polynomial time	22
8.5 왜 graph connectivity 예제가 자주 나오는가?	22
9 Automata shortcut: sequential algorithm을 꼭 순차적으로 풀어야 하나?	23
9.1 Liu et al.: shallow Transformer가 automaton을 shortcut 할 수 있다	23
9.2 Binary-tree composition	23
9.3 이 결과와 TC^0 upper bound는 충돌하지 않는다	23
9.4 Learning과 existence를 다시 분리한다	24
10 Formal language hierarchy로 결과를 읽는 법	25
10.1 대표 언어 세 개	25
10.2 “Regular language를 못한다”는 표현을 조심해야 하는 이유	25
10.3 Turing completeness와 formal-language upper bound를 동시에 기억하는 법	26
11 Arithmetic와 algorithmic reasoning을 어떻게 해석할까?	27
11.1 TC^0 안에도 강한 arithmetic이 있다	27
11.2 어려움은 arithmetic operation 자체보다 sequential dependency일 수 있다	27
11.3 CoT가 제공하는 것은 externalized recurrence	27
11.4 그러면 내부 hidden reasoning은 의미가 없는가?	28
12 결과를 과장하지 않기 위한 수학적 체크리스트	29
12.1 Turing complete $\neq$ efficient	29
12.2 TC^0 upper bound $\neq$ empirical impossibility	29
12.3 CoT power $\neq$ faithful reasoning	29
12.4 Architecture assumption을 반드시 붙인다	29
12.5 세미나에서 가장 중요한 conceptual distinction	30
13 발표 준비용 압축 정리	31
13.1 칠판에 반드시 남길 열 개 식	31
13.2 발표 순서 추천	31
13.3 자주 나오는 질문	32
13.4 혼한 수학적 실수 체크리스트	32
13.5 30초 결론	33
참고문헌 및 원문에서 확인할 위치	34

1

먼저 맞춰 들 기호와 전체 그림

1.1 이번 주에 섞이면 안 되는 네 개의 축

Transformer computational theory에서 가장 혼한 혼동은 서로 다른 resource를 한 문장에 섞는 것이다. 같은 “Transformer”라는 단어라도 theorem마다 다음 네 가지가 다르다.

축	대표 선택지	왜 중요한가
Attention	soft / hard / saturated	exact addressing, averaging, circuit simulation 가능성이 달라진다.
Precision	$O(1)$ / $O(\log n)$ / arbitrary	한 scalar가 저장할 수 있는 정보량이 달라진다.
Sequential steps	one pass / $t(n)$ decoding steps	fixed-depth parallel 계산인지, 반복적인 순차 계산인지가 달라진다.
Architecture scaling	depth/width fixed vs. n 에 따라 증가	“하나의 모델”인지 input length마다 더 큰 회로를 허용하는지 결정한다.

핵심 “Transformer는 Turing complete다”와 “Transformer는 TC^0 를 넘지 못한다”는 문장은 동시에 참일 수 있다. 앞 문장은 arbitrary precision, hard attention, unbounded decoding을 허용하는 계산 모델에 관한 결과이고, 뒤 문장은 log precision과 고정된 forward-pass depth를 가진 모델에 관한 upper bound다.

1.2 논문별 기호 대응표

이 노트	Pérez et al.	Merrill–Sabharwal	의미
n	$ w $	n	input length
L	encoder/decoder layer 수	d	Transformer depth
H	attention head 수	h	number of heads
$p(n)$	rational precision	p	arithmetic precision in bits
$t(n)$	output length / TM steps	$t(n)$	intermediate decoding steps
Σ	Σ	Σ	finite alphabet
$\mathcal{L}$	L	L	formal language
M	$M = (Q, \Sigma, \delta, q_{\text{init}}, F)$	Turing machine M	simulated machine

1.3 결론을 먼저 보는 지도

이번 주 결과를 resource 관점에서 한 줄씩 적으면 다음과 같다.

hard/saturated, fixed depth $\rightsquigarrow \text{AC}^0$ 또는 TC^0 upper bound,
 log precision, one forward pass $\rightsquigarrow$ uniform TC^0 ,
 arbitrary precision + hard attention $\rightsquigarrow$ Turing completeness with unbounded decoding,
 $t(n)$ autoregressive steps $\rightsquigarrow t(n)$ 이 순차 계산 resource가 됨.

따라서 이번 주의 핵심 질문은 architecture 이름이 아니라

무엇을 고정하고, 무엇을 input length와 함께 증가시키는가?

이다.

발표에서 이렇게 말하기 “이번 주에는 Transformer의 계산력을 하나의 숫자로 말하지 않겠습니다. Attention type, numerical precision, model depth, 그리고 autoregressive step 수를 따로 잡아야 theorem들이 서로 맞아떨어집니다. 특히 one-pass Transformer와 CoT를 쓰는 decoder는 계산 모델 자체가 다릅니다.”

2

Formal language와 circuit complexity의 최소 배경

2.1 Language recognition을 계산 문제로 보기

정의 2.1 (Formal language). 유한 alphabet Σ 에 대해

$$\Sigma^* = \bigcup_{n \geq 0} \Sigma^n$$

이고, formal language는 임의의 부분집합

$$\mathcal{L} \subseteq \Sigma^*$$

이다.

모델이 문자열 $x \in \Sigma^*$ 를 받아

$$f(x) \in \{0, 1\}$$

을 출력하고

$$f(x) = 1 \iff x \in \mathcal{L}$$

이면 f 가 $\mathcal{L}$ 을 recognize한다고 한다.

이 정의는 language modeling probability와 다르다. 여기서는 특정 distribution 위 평균 loss가 아니라 모든 길이의 모든 입력에 대해 **membership**을 정확히 계산할 수 있는지를 본다.

2.2 Finite automaton, regular language, 그리고 sequentiality

DFA는

$$A = (Q, \Sigma, \delta, q_0, F)$$

로 쓰며 입력

$$x = x_1 x_2 \cdots x_n$$

에 대해

$$q_i = \delta(q_{i-1}, x_i), \quad i = 1, \dots, n$$

을 반복한다.

최종적으로

$$x \in \mathcal{L}(A) \iff q_n \in F.$$

즉 DFA 자체는 아주 작은 memory만 필요하지만 계산 graph는 길이 n 의 recurrence다. Transformer computational theory에서 이것이 중요한 이유는 “작은 memory”와 “작은 parallel depth”가 같은 개념이 아니기 때문이다.

보충 유도 — 논문 밖의 독자용 전개 Transition을 함수 합성으로 쓰면

$$q_n = \delta_{x_n} \circ \delta_{x_{n-1}} \circ \cdots \circ \delta_{x_1}(q_0), \quad \delta_a(q) := \delta(q, a).$$

순차적으로는 n 번 적용하지만, 함수 합성이 결합법칙을 가지므로 parallel prefix 방식으로 tree를 만들면 $O(\log n)$ depth로도 계산 가능하다. 이 관점이 이후 automata shortcut과 NC^1 연결의 직관이다.

2.3 Circuit family: input length마다 회로 하나

Boolean circuit C_n 은 길이 n 인 bit string을 입력받는다. 모든 길이를 처리하려면

$$\{C_n\}_{n \geq 1}$$

이라는 circuit family를 생각한다.

두 resource가 핵심이다.

- **size:** gate/wire의 총 개수,
- **depth:** input에서 output까지 가장 긴 dependency chain.

Transformer layer 수가 input length에 무관한 상수라면, forward pass를 회로로 펼쳤을 때 자연스럽게 constant-depth computation과 비교하게 된다.

2.4 AC^0 , TC^0 , NC^1

정의 2.2 (AC^0). Polynomial-size, constant-depth Boolean circuit family이며 AND, OR, NOT gate가 허용된다. AND/OR의 fan-in은 unbounded일 수 있다.

정의 2.3 (TC^0). AC^0 에 unbounded-fan-in threshold/majority gate를 허용한 클래스다. 대표 threshold gate는

$$\text{THR}(x_1, \dots, x_m) = \mathbf{1} \left[\sum_{i=1}^m w_i x_i \geq \theta \right].$$

표준 uniform 버전을 염두에 두면 다음 containment를 기억하면 충분하다.

$$AC^0 \subsetneq TC^0 \subseteq NC^1 \subseteq L \subseteq NL \subseteq P.$$

첫 번째 strictness는 parity/majority에 대한 고전적 lower bound에서 오지만, TC^0 와 NC^1 사이의 strictness 같은 더 뒤의 분리는 알려져 있지 않다. 따라서 논문은 보통

$$TC^0 \neq NC^1, \quad L \neq P$$

같은 표준 complexity conjecture를 조건으로 “못 푼다”는 결론을 말한다.

2.5 왜 TC^0 가 생각보다 약하지 않은가

TC^0 는 constant depth라고 해서 단순한 함수만 포함하는 것은 아니다. threshold gate 덕분에 다수결, 정수 덧셈, 곱셈과 같은 상당히 풍부한 arithmetic이 매우 parallel하게 구현될 수 있다.

따라서

$$\text{“Transformer} \in TC^0\text{”}$$

라는 upper bound는 “덧셈도 못한다”가 아니라 즉 이 upper bound의 해석은

“본질적으로 긴 sequential dependency가 필요한 일부 문제에서는 한 번의 고정-depth pass가 제한을 받는다.”

에 가깝다.

주의 Complexity class inclusion은 실제 LLM accuracy에 대한 직접 예측이 아니다. Theorem의 모델은 precision, activation, masking, uniformity를 엄밀히 고정한 계산 모델이다. 실제 finite-length benchmark에서 특정 task를 근사적으로 맞추는 것과 모든 n 에 대해 exact recognition하는 것은 다른 주장이다.

발표에서 이렇게 말하기 “Formal language를 쓰는 이유는 자연어가 정규언어라는 주장을 하려는 게 아닙니다. 입력 길이가 커질 때 모델이 parity, bracket matching, automaton simulation 같은 계산을 정확히 수행할 수 있는지를 clean하게 측정하기 위해서입니다. 그리고 circuit complexity는 Transformer의 강한 parallelism을 depth라는 자원으로 번역해 줍니다.”

3

고정-depth self-attention의 초기 한계

3.1 Hahn의 질문: recurrence 없이 hierarchy를 처리할 수 있는가?

Hahn (2020)은 고정된 layer/head 수의 self-attention 모델이 input length가 증가할 때 formal language를 어떻게 처리하는지 분석했다. 대표 예는 다음 두 계산이다.

- **Parity:** $x \in \{0, 1\}^n$ 에 대해

$$\text{PARITY}(x) = \left(\sum_{i=1}^n x_i \right) \bmod 2.$$

- **Dyck language:** 괄호 prefix마다 열린 괄호 수가 음수가 되지 않고 마지막에 0이 되는지 검사한다.

Parity는 finite-state computation이지만 모든 bit의 global interaction이 필요하고, Dyck는 unbounded hierarchical memory를 요구한다. Hahn의 결과는 fixed-size self-attention이 이런 구조를 길이에 무관하게 exact하게 처리하는 데 강한 제약이 있음을 보였다.

원 논문 기준— **Hahn (2020)** Soft attention과 hard attention 모두에서, model의 layer 수나 head 수를 input length에 따라 키우지 않는 한 periodic finite-state language와 hierarchical structure를 일반적으로 처리할 수 없다는 한계가 제시된다. 이 결과는 이후 circuit-complexity upper bound로 더 구조적으로 정리된다.

3.2 Hard attention을 회로로 보면 더 깔끔해진다

Hao, Angluin, Frank (2022)는 hard-attention Transformer를 세 종류로 나눈다.

- **UHAT:** unique hard attention,
- **GUHAT:** UHAT의 generalized version,
- **AHAT:** maximum score 위치들을 평균하는 averaging hard attention.

핵심 upper bound는

$$\text{UHAT, GUHAT} \subseteq \text{AC}^0$$

이다.

AC^0 는 parity를 recognize할 수 없으므로 즉시

$$\text{PARITY} \notin \mathcal{L}(\text{GUHAT})$$

같은 결론이 따른다.

반면 AHAT는 tie된 여러 위치를 평균할 수 있어 더 강해지고, 논문에서는 MAJORITY와 DYCK-1 같은 non- AC^0 language를 recognize하는 construction을 보인다.

3.3 “attention이 global이니 sequential problem도 쉽다”가 왜 틀릴 수 있는가

Self-attention은 한 층에서 모든 token pair를 연결한다.

$$a_i = \sum_{j=1}^n \alpha_{ij} v_j.$$

하지만 dependency graph가 dense하다는 사실과 arbitrary sequential algorithm을 구현할 수 있다는 사실은 다르다. 한 층은 모든 값을 볼 수 있지만, 여러 단계의 state transition을 차례로 적용해야 하는 계산은 별도의 depth 또는 recurrence가 필요할 수 있다.

예를 들어 DFA transition을

$$q_n = \delta_{x_n} \circ \dots \circ \delta_{x_1}(q_0)$$

로 쓰면, 모든 x_i 를 한 번에 읽는 것과 이 ordered composition을 계산하는 것은 다른 문제다.

핵심 Attention의 “global receptive field”는 communication distance를 줄여 주지만, computational depth를 자동으로 늘려 주지 않는다. 이번 주 이론에서 가장 중요한 구분 중 하나다.

발표에서 이렇게 말하기 “한 층의 attention은 모든 토큰을 볼 수 있지만, 모든 알고리즘을 한 단계에 수행할 수 있다는 뜻은 아닙니다. Circuit 관점에서는 communication은 global이어도 layer 수가 고정되어 있으면 computational depth는 여전히 상수입니다. 그래서 parity나 automaton composition 같은 문제가 중요한 test case가 됩니다.”

4

Log-precision Transformer와 TC^0 upper bound

4.1 왜 precision을 명시해야 하는가?

실수 하나에 arbitrary precision을 허용하면 이론적으로 무한히 많은 정보를 한 scalar에 숨길 수 있다. 따라서 실제 digital computation에 가까운 모델을 만들려면 input length n 에 따라 사용할 수 있는 bit 수를 제한해야 한다.

Merrill–Sabharwal (2023)의 핵심 setting은

$$p(n) = O(\log n)$$

bit precision이다.

이 정도 precision이면 값의 표현 범위와 resolution이 polynomial scale에 머물면서도, 길이 n 의 sequence 위 uniform averaging 같은 attention operation을 표현할 수 있다.

4.2 Main theorem

정리 4.1 (Merrill–Sabharwal 2023, informal). Arithmetic precision이 $O(\log n)$ 이고 feed-forward subnetwork가 입력 크기에 선형인 space에서 계산 가능한 Transformer classifier는 constant-depth logspace-uniform threshold circuit로 simulate할 수 있다. 따라서

$$\text{log-precision fixed-depth Transformer} \subseteq \text{uniform } \text{TC}^0.$$

원 논문 기준 이 theorem에서 “Transformer”는 input length가 커져도 layer 수, head 수, hidden architecture가 고정된 family를 의미한다. 핵심은 한 번의 forward pass를 constant-depth threshold circuit로 compile할 수 있다는 것이다.

4.3 왜 circuit depth가 constant 인가? — proof skeleton

Transformer layer를 대략

$$H^{(\ell+1)} = \text{FFN}^{(\ell)} \left(H^{(\ell)} + \text{Attn}^{(\ell)}(H^{(\ell)}) \right)$$

라고 하자.

전체 layer 수가 $L = O(1)$ 이면 다음을 보이면 충분하다.

1. embedding과 positional feature가 TC^0 에서 계산 가능,
2. attention score 계산이 TC^0 에서 가능,
3. n 개의 값의 합/정규화가 TC^0 에서 가능,

4. FFN arithmetic 이 TC^0 에서 가능,
5. residual/normalization 도 TC^0 에서 가능.

각 block 이 constant-depth threshold circuit 이고 layer 수 L 도 constant 이면 합성한 depth 는

$$D_{\text{total}} \leq L D_{\text{layer}} + D_{\text{embed}} + D_{\text{out}} = O(1).$$

또 각 위치 $i = 1, \dots, n$ 의 계산을 parallel 하게 복제하므로 전체 gate 수는 polynomial size 에 머문다.

보충 유도 — 논문 밖의 독자용 전개 왜 $O(\log n)$ precision 이 polynomial-size circuit 와 잘 맞는 지 보자. $p = c \log n$ bit 이면 한 scalar 는 최대

$$2^p = n^c$$

개의 bit pattern 수준으로 표현된다. 따라서 arithmetic operand 의 bit-length 는 input length 에 대해 logarithmic 한 범위에 머문다.

이 때문에 polynomial-size circuit simulation 을 유지할 수 있다. 반대로 arbitrary precision 을 허용하면 scalar 하나가 input length 에 비해 과도하게 많은 정보를 저장할 수 있어 이 argument 가 깨진다.

4.4 Self-attention의 합이 왜 “순차 합산”을 요구하지 않는가?

attention output 은

$$a_i = \frac{\sum_{j=1}^n s_{ij} v_j}{\sum_{j=1}^n s_{ij}}$$

형태다. 프로그램으로 쓰면 $j = 1$ 부터 n 까지 loop 를 도는 것처럼 보인다. 하지만 circuit model 에서는 n 개 항을 parallel tree 또는 threshold arithmetic circuit 로 합산할 수 있다.

따라서 wall-clock sequential code 와 circuit depth 를 혼동하면 안 된다. Transformer 의 parallelism 이 바로 이 지점에서 complexity upper bound 에 반영된다.

4.5 Upper bound의 실제 의미

TC^0 upper bound 로부터 바로 “특정 문제는 불가능”이라고 말하려면 complexity separation 이 필요하다. 예를 들어 paper 는 표준 가정 아래 다음 식의 해석을 강조한다.

one-pass Transformer $\in TC^0$ vs. problem complete for a larger class.

예를 들어 arbitrary finite-state machine simulation 은 NC^1 수준의 sequential composition 과 연결되고, directed graph connectivity 는 NL-complete, 일부 algebraic reasoning problem 은 P-complete 하다. 따라서 적절한 class non-collapse assumption 아래 fixed-depth one-pass Transformer 의 한계가 된다.

주의 “ TC^0 upper bound 이므로 모든 regular language 를 못 푼다”라고 무조건 쓰면 안 된다. 많은 regular language 는 AC^0 나 TC^0 안에 있다. 정확한 주장은, 표준 separation 가정 아래 모든 regular language 를 한 architecture class 가 처리할 수는 없다는 식으로 해야 한다.

발표에서 이렇게 말하기 “Log-precision theorem의 요지는 attention 연산 자체가 약하다는 게 아닙니다. 한 forward pass가 엄청나게 parallel하기 때문에, 그 계산 전체가 constant-depth threshold circuit로 compile 됩니다. 이 parallelism이 장점인 동시에 sequential depth를 제한하는 원인이 됩니다.”

5

그런데 왜 Transformer가 Turing complete 인가?

5.1 Turing completeness의 정의부터 분리한다

정의 5.1 (Turing completeness, language-recognition view). 계산 모델의 class $\mathcal{N}$ 이 모든 Turing machine이 결정할 수 있는 language를 simulate할 수 있으면 $\mathcal{N}$ 을 Turing complete라고 한다.

중요한 것은 Turing completeness가

“고정된 한 번의 forward pass로 모든 계산을 끝낸다”

는 뜻이 아니라는 점이다. Turing machine도 problem size에 따라 임의로 많은 step을 사용한다.

5.2 Pérez–Barceló–Marinkovic의 main theorem

정리 5.1 (Pérez et al. 2021, Theorem 6). Positional encoding을 가진 hard-attention Transformer class는 Turing complete하다. 논문의 construction에서는 positional embedding의 non-constant coordinate로

$$n, \quad \frac{1}{n}, \quad \frac{1}{n^2}$$

만 사용하며, single encoder layer와 three decoder layers로도 충분하다.

논문은 더 강하게 recursively enumerable language까지 recognize할 수 있는 construction을 논의한다.

원 논문 기준— theorem의 중요한 가정 증명은 rational-valued **arbitrary precision** internal representation과 **hard attention**을 사용한다. 저자들도 fixed precision이면 같은 Turing-completeness conclusion이 성립하지 않는다고 명시한다.

5.3 TM simulation의 상태를 무엇으로 저장하는가?

한 tape Turing machine의 시각 t configuration을 개념적으로

$$C_t = (q_t, h_t, \tau_t)$$

로 쓰자.

- q_t : finite control state,
- $h_t \in \mathbb{Z}$: head position,
- $\tau_t : \mathbb{Z} \rightarrow \Sigma$: tape content.

transition function은

$$\delta(q_t, \tau_t(h_t)) = (q_{t+1}, s_t^{\text{write}}, d_t), \quad d_t \in \{-1, +1\}$$

이고

$$h_{t+1} = h_t + d_t.$$

문제는 tape 전체 τ_t 를 hidden vector 하나에 넣는 것이 아니다. Decoder가 지금까지 생성한 history를 memory처럼 사용하고, hard attention으로 필요한 위치를 retrieve한다.

5.4 가장 최근 write를 찾으려면 tape를 읽을 수 있다

현재 head가 cell $j = h_t$ 를 읽는다고 하자. 그 cell에 마지막으로 write한 시점을

$$r_t(j) := \max\{s < t : h_s = j \text{ and step } s \text{ writes to } j\}$$

로 두자.

그러면 현재 tape symbol은

$$\tau_t(j) = \begin{cases} \text{symbol written at step } r_t(j), & r_t(j) \text{가 존재,} \\ \text{initial symbol at } j, & \text{otherwise.} \end{cases}$$

즉 decoder history 안에서

“현재 cell과 같은 address를 가진 가장 최근 event”

만 찾으면 된다. Pérez et al.의 hard-attention construction은 positional rational code를 이용해 이런 addressing을 구현한다.

보충 유도 — 논문 밖의 독자용 전개 이 construction을 database 관점으로 보면 이해가 쉽다. 각 decoding token은

(time, head address, write symbol, state)

를 기록하는 log entry다. 현재 step은 attention으로 과거 log 중 필요한 address의 최신 record를 읽고, FFN이 finite transition δ 를 적용해 다음 log entry를 쓴다. Autoregressive generation이 곧 Turing-machine time axis가 된다.

5.5 Positional encoding이 왜 핵심인가?

Pérez et al.은 positional encoding이 없는 Transformer가 강한 proportion invariance를 가진다는 것을 먼저 보인다. 두 입력 u, w 가 각 symbol의 비율을 같게 가지면 특정 설정에서 Transformer output이 같아지는 제약이 생긴다.

따라서 순서와 정확한 address를 다루려면 position 정보를 제공해야 한다. Construction에서는 $n, 1/n, 1/n^2$ 형태를 사용해 decoder step과 tape address를 분리하고 비교할 수 있게 한다.

5.6 Fixed precision이면 왜 같은 증명이 무너지는가?

Arbitrary precision에서는 position index n 이 아무리 커져도 서로 다른 rational code를 유지할 수 있다. 하지만 fixed precision에서는 positional value가 취할 수 있는 값의 집합이 finite다.

충분히 긴 sequence에서는 pigeonhole principle에 의해 서로 다른 position들이 같은 code로 충돌한다. 따라서 unbounded tape address를 exact하게 encode하는 Pérez-style proof가 유지되지 않는다.

핵심 Turing completeness theorem의 “infinite computation”은 hidden dimension이 무한해서가 아니다. 고정된 architecture가 arbitrary number of autoregressive steps와 arbitrary-precision numerical state를 사용해 Turing machine trajectory를 계속 이어 갈 수 있다는 뜻이다.

발표에서 이렇게 말하기 “Turing-complete proof에서 Transformer는 한 번에 Turing machine 전체를 계산하지 않습니다. Decoder가 한 token을 생성할 때마다 machine의 한 step을 진행하고, 이미 생성된 prefix를 외부 memory처럼 읽습니다. 그래서 이 결과를 one-pass TC^0 upper bound와 비교하려면 decoding time과 precision을 반드시 같이 봐야 합니다.”

6

모순 해소: one-pass TC^0 와 Turing completeness

6.1 두 theorem의 assumptions를 같은 표에 놓기

항목	Log-precision upper bound	Pérez Turing completeness
Precision	$O(\log n)$ bits	arbitrary-precision rationals
Attention	practical-style arithmetic / bounded precision	hard attention essential to proof
Sequential compute	one fixed-depth forward pass	unbounded autoregressive decoder steps
Position information	bounded representation	exact rational positional codes
Conclusion	uniform TC^0 upper bound	arbitrary TM simulation 가능

둘은 서로 다른 계산 모델이므로 논리적 충돌이 없다.

6.2 계산 resource를 식으로 분해하면 더 명확하다

Transformer decoder의 전체 계산량을 아주 거칠게

$$\text{Total sequential depth} \approx \underbrace{L}_{\text{layers per pass}} \times \underbrace{T}_{\text{autoregressive passes}}$$

로 생각할 수 있다.

one-pass theorem에서는

$$L = O(1), \quad T = O(1).$$

Turing-machine simulation에서는 architecture depth L 은 고정이어도

$$T = T(n)$$

가 input-dependent하게 커질 수 있다.

따라서 모델 parameter 수는 같아도 실행 시간이 계산력을 증가시킨다.

6.3 Precision도 독립적인 resource다

또 하나의 축은 bit precision이다.

$$\text{information per scalar} \sim p(n) \text{ bits.}$$

$O(\log n)$ precision은 polynomial range/addressing에 충분하지만 unbounded exact position encoding을 scalar 하나에 계속 넣을 수는 없다. Arbitrary precision은 이 restriction을 없앤다.

따라서 computational power를 비교할 때 최소한

$$(L, H, d, p(n), T(n))$$

를 함께 적는 습관이 필요하다.

주의 Turing completeness는 실용적 학습 가능성, numerical robustness, finite precision implementation을 보장하지 않는다. 반대로 TC^0 upper bound도 finite benchmark에서 LLM이 heuristic이나 approximate solution으로 높은 accuracy를 낼 수 없다는 뜻이 아니다.

발표에서 이렇게 말하기 “겉으로는 한쪽 논문이 Turing complete, 다른 논문이 TC^0 라고 해서 충돌해 보입니다. 하지만 계산 모델을 (precision, depth, decoding steps)로 쓰면 바로 풀립니다. Turing-complete construction은 sequential time과 arbitrary precision을 자원으로 쓰고, TC^0 theorem은 그 자원을 제한한 one-pass 모델입니다.”

7

Chain of Thought를 계산 resource로 보기

7.1 Intermediate token 하나는 forward pass 하나다

Decoder-only Transformer를

$$f : \Sigma^* \rightarrow \Sigma$$

라고 하자. 한 번의 decoding으로

$$y_1 = f(x)$$

를 만들고 다시 prefix에 붙이면

$$y_2 = f(xy_1), \quad \dots, \quad y_t = f(xy_1 \cdots y_{t-1}).$$

즉 CoT/scratchpad 길이 t 는 단순히 “답을 길게 쓰는 것”이 아니라

t 번의 serial forward passes

를 허용하는 resource다.

정의 7.1 ($\text{CoT}(t(n))$). 입력 길이 n 에 대해 최대 $t(n)$ 개의 intermediate decoding step을 사용하는 어떤 Transformer decoder가 recognize할 수 있는 language들의 class를 $\text{CoT}(t(n))$ 로 쓴다.

7.2 Merrill–Sabharwal의 central sandwich

ICLR 2024 결과를 핵심 형태로 쓰면, polylog factor를 숨겼을 때

$$\text{TIME}(t(n)) \subseteq \text{CoT}(t(n)) \subseteq \text{SPACE}(t(n) + \log n) \subseteq \widetilde{\text{TIME}}(t(n)^2 + n^2)$$

이다.

여기서 왼쪽 lower bound construction은 strict causal saturated attention과 projected pre-norm을 사용한다. Upper bound는 그 특수 construction에 의존하지 않는다.

7.3 왼쪽 inclusion: TM step 하나를 token 하나로

$$\text{TIME}(t(n)) \subseteq \text{CoT}(t(n))$$

의 직관은 Pérez construction과 비슷하지만, log-precision decoder-only setting에 맞게 더 정교하게 만든 것이다.

Turing machine의 transition record를

$$\Delta_i = (q_i, \text{symbols read}, \text{symbols written}, \text{head moves})$$

라고 하자.

Construction은 induction으로

$$\text{decoding step } i \mapsto \Delta_i$$

를 생성한다.

현재 tape cell의 값을 알아내려면 가장 최근에 같은 cell에 write한 token을 찾아야 한다. 이를 위해 논문은 layer-norm hash를 사용한다.

7.4 Layer-norm hash를 한 줄씩 유도

논문의 핵심 device를 직접 계산해 보자.

$$v_x := \langle x, 1, -x, -1 \rangle.$$

평균은 0이고

$$\|v_x\|^2 = x^2 + 1 + x^2 + 1 = 2x^2 + 2.$$

따라서 normalized vector를

$$\phi_x := \frac{1}{\sqrt{2x^2 + 2}} \langle x, 1, -x, -1 \rangle$$

로 둔다.

이제 scale된 pair $(x/i, 1/i)$ 로 같은 vector를 만들면

$$\begin{aligned} \phi(x/i, 1/i) &= \frac{\langle x/i, 1/i, -x/i, -1/i \rangle}{\sqrt{2x^2/i^2 + 2/i^2}} \\ &= \frac{(1/i) \langle x, 1, -x, -1 \rangle}{(1/i) \sqrt{2x^2 + 2}} \\ &= \phi_x. \end{aligned}$$

즉 layer normalization이 공통 scale $1/i$ 를 지워 주어 position-dependent denominator 속에서 discrete address x 를 복원한다.

두 hash의 inner product는

$$\begin{aligned} \langle \phi_q, \phi_k \rangle &= \frac{qk + 1 + qk + 1}{\sqrt{(2q^2 + 2)(2k^2 + 2)}} \\ &= \frac{qk + 1}{\sqrt{(q^2 + 1)(k^2 + 1)}}. \end{aligned}$$

언제 이 값이 최대 1이 되는가?

$$\begin{aligned} 1 &= \frac{qk + 1}{\sqrt{(q^2 + 1)(k^2 + 1)}} \\ (qk + 1)^2 &= (q^2 + 1)(k^2 + 1) \\ q^2k^2 + 2qk + 1 &= q^2k^2 + q^2 + k^2 + 1 \\ 0 &= q^2 - 2qk + k^2 \\ 0 &= (q - k)^2. \end{aligned}$$

따라서

$$\boxed{\langle \phi_q, \phi_k \rangle = 1 \iff q = k.}$$

Attention score로 이 inner product를 쓰면 exact equality lookup을 구현할 수 있다.

원 논문 기준 이 layer-norm hash는 ICLR 2024 paper의 proof에서 핵심 construction이다. Projected pre-norm은 hidden state의 필요한 coordinate subset만 normalize해 이런 hash를 여러 용도로 만들 수 있게 한다.

7.5 Regular language는 왜 linear CoT면 충분한가?

DFA state를 q_i 라고 하면

$$q_i = \delta(q_{i-1}, x_i).$$

Input을 다 읽은 뒤 intermediate token에서

$$q_0, q_1, \dots, q_n$$

을 차례로 생성하면 된다.

각 decoding step에서는

1. 이전 state q_{i-1} 를 current prefix에서 읽고,
2. layer-norm hash로 x_i 에 해당하는 input position을 찾고,
3. FFN이 finite transition table δ 를 계산한다.

따라서 모든 regular language $\mathcal{L}$ 에 대해 적절한 projected-pre-norm decoder가

$$|x| + 1$$

decoding steps로 membership을 판정할 수 있다.

7.6 Polynomial CoT가 왜 P까지 가는가?

Lower bound 쪽에서는 polynomial-time Turing machine의 각 step을 한 decoding token으로 simulate하므로

$$P \subseteq \text{CoT}(\text{poly}(n)).$$

Upper bound 쪽에서는 polynomial number of forward passes 각각이 polynomial-time에 simulate 가능하므로

$$\text{CoT}(\text{poly}(n)) \subseteq P.$$

따라서 해당 architectural assumptions 아래

$$\boxed{\text{CoT}(\text{poly}(n)) = P.}$$

이것은 “LLM이 P-complete problem을 실제로 학습한다”는 theorem이 아니라, allowed computation class의 exact characterization이다.

발표에서 이렇게 말하기 “CoT의 계산이론적 의미는 intermediate text의 영어 의미가 아닙니다. 중요한 건 token 하나를 쓸 때마다 Transformer를 다시 한 번 실행한다는 사실입니다. 그래서 scratchpad 길이가 serial time budget이 되고, linear CoT부터 automaton을 직접 simulation할 수 있으며 polynomial CoT에서는 P까지 올라갑니다.”

8

CoT 길이에 따른 계산력의 계층

8.1 $O(1)$: intermediate generation이 없는 경우

Log-precision fixed-depth Transformer 한 번의 출력은

$$\text{CoT}(O(1)) \subseteq \text{TC}^0$$

관점에서 이해할 수 있다.

따라서 standard separation assumption 아래 automaton simulation처럼 NC^1 -complete한 sequential composition 문제를 모두 처리할 수 없다.

8.2 $O(\log n)$: 조금의 scratchpad는 얼마나 도움이 되는가?

Space upper bound

$$\text{CoT}(t(n)) \subseteq \text{SPACE}(t(n) + \log n)$$

예

$$t(n) = O(\log n)$$

을 대입하면

$$\text{CoT}(O(\log n)) \subseteq \text{SPACE}(O(\log n)) = \text{L}.$$

즉 logarithmic CoT는 TC^0 보다 약간 더 많은 sequentiality를 줄 수 있지만, 여전히 logspace 범위 안에 있다.

8.3 $\Theta(n)$: 모든 regular language

Theorem 1의 construction으로

$$\text{REG} \subseteq \text{CoT}(\Theta(n)).$$

동시에 space upper bound는

$$\text{CoT}(\Theta(n)) \subseteq \text{SPACE}(O(n)).$$

Linear bounded space로 recognize되는 language가 context-sensitive language class와 연결되므로

$$\text{CoT}(\Theta(n)) \subseteq \text{CSL}$$

라는 upper-bound picture를 얻는다.

8.4 $\text{poly}(n)$: 정확히 polynomial time

앞 절에서 본 것처럼

$$\text{CoT}(\text{poly}(n)) = \text{P}$$

가 된다.

이를 한 줄 표로 정리하면 다음과 같다.

Intermediate steps	알려진 계산력 그림	해석
$O(1)$	$\subseteq \text{uniform TC}^0$	highly parallel one-pass computation
$O(\log n)$	$\subseteq \text{L}$	logspace까지의 작은 sequential workspace
$\Theta(n)$	$\text{REG} \subseteq \text{CoT}(n) \subseteq \text{CSL}$	DFA simulation 가능
$\text{poly}(n)$	$= \text{P}$ (해당 construction assumptions)	polynomial-time TM simulation

핵심 CoT의 이론적 효과는 “reasoning style”이 아니라 **adaptive sequential depth**를 추가한다는 것이다. 어려운 문제에 더 많은 token을 쓰면 더 많은 serial computation을 배정할 수 있다.

8.5 왜 graph connectivity 예제가 자주 나오는가?

Directed graph reachability는 대표적인 NL-complete problem이다. One-pass TC^0 upper bound와는 큰 gap이 있지만, 충분한 intermediate step을 쓰면 DFS/BFS류 sequential algorithm을 simulate할 수 있다.

ICLR 2024 paper는 random-access가 없는 Turing machine simulation overhead를 고려해 directed connectivity에 quadratic step upper bound를 예시로 든다. 핵심은 구체적인 n^2 상수보다

scratchpad가 graph traversal의 serial state를 저장할 수 있다

는 점이다.

발표에서 이렇게 말하기 “CoT length를 resource axis로 보면 깔끔한 hierarchy가 생깁니다. 상수 step은 TC^0 , 로그 step은 logspace upper bound, 선형 step부터 모든 finite automaton을 simulation하고, polynomial step에서는 polynomial-time Turing machine과 맞아집니다. 이게 CoT를 단순 prompting trick이 아니라 compute allocation으로 볼 수 있는 이론적 근거입니다.”

9

Automata shortcut: sequential algorithm을 꼭 순차적으로 풀어야 하나?

9.1 Liu et al.: shallow Transformer가 automaton을 shortcut할 수 있다

앞 절에서는 DFA를 token-by-token simulate하려면 $O(n)$ CoT step이면 충분하다고 했다. 그렇다면 CoT 없이 shallow Transformer는 automaton 계산을 전혀 못하는가?

Liu et al. (ICLR 2023)은 더 미묘한 답을 준다. Automaton transition들의 algebraic structure를 이용하면 recurrent trajectory를 직접 따라가지 않고도 parallel하게 composition할 수 있다. 모든 automaton에 대해 polynomial-size

$$O(\log n)$$

depth Transformer simulator가 존재하며, 특정 algebraic structure를 가진 automata에는 $O(1)$ -depth shortcut도 존재할 수 있다.

9.2 Binary-tree composition

입력 각 symbol이 state-transition function

$$f_i : Q \rightarrow Q, \quad f_i(q) = \delta(q, x_i)$$

를 정한다고 하자.

원래 계산은

$$q_n = (f_n \circ f_{n-1} \circ \cdots \circ f_1)(q_0).$$

함수 합성은 associative하므로

$$(f_4 \circ f_3) \circ (f_2 \circ f_1)$$

같은 balanced tree로 계산할 수 있다.

Level 1에서 길이-2 block의 composition, level 2에서 길이-4 block, ..., level $\lceil \log_2 n \rceil$ 에서 전체 composition을 얻는다.

따라서 recurrent time n 을

$$O(\log n) \text{ parallel depth}$$

로 줄일 수 있다는 직관이 나온다.

9.3 이 결과와 TC^0 upper bound는 충돌하지 않는다

$O(\log n)$ depth는 constant depth가 아니다. Input length에 따라 Transformer layer 수 자체를 키우는 family를 허용하므로 one-pass fixed-depth TC^0 theorem과 setting이 다르다.

또 일부 automaton은 constant-depth shortcut을 갖지만, 모든 automaton에 대해 그런 shortcut이 존재하는 것은 아니다.

9.4 Learning과 existence를 다시 분리한다

Liu et al.의 실험은 synthetic automata에서 training이 shortcut solution을 찾을 수 있음을 보여주지만, 이런 solution이 length extrapolation에서 brittle할 수 있음을 함께 지적한다.

따라서

existence of shallow circuit $\nRightarrow$ robust algorithmic generalization.

이 구분은 Week 4의 UAT와도 같은 패턴이다. 표현 가능성 theorem과 실제 optimization/generalization theorem은 다르다.

발표에서 이렇게 말하기 “Sequential algorithm이라고 해서 반드시 n layer가 필요한 것은 아닙니다. Automaton transition은 associative composition이므로 tree reduction으로 $O(\log n)$ depth shortcut을 만들 수 있습니다. 다만 그건 layer 수가 input length와 함께 증가하는 construction이고, 학습된 shortcut이 길이 밖에서도 robust하다는 보장은 별도 문제입니다.”

10

Formal language hierarchy로 결과를 읽는 법

10.1 대표 언어 세 개

Parity

$$\mathcal{L}_{\text{parity}} = \left\{ x \in \{0, 1\}^* : \sum_i x_i \equiv 1 \pmod{2} \right\}.$$

Regular language지만 AC^0 밖에 있다. 따라서 “regular”이 곧 “constant-depth AND/OR circuit로 쉽다”는 뜻은 아니다.

Dyck-1

한 종류의 괄호에 대해 prefix balance

$$b_k = \sum_{i=1}^k \begin{cases} +1, & x_i = (, \\ -1, & x_i =), \end{cases}$$

가

$$\min_{k \leq n} b_k \geq 0, \quad b_n = 0$$

을 만족하는 language다. Unbounded nesting 때문에 context-free structure의 대표 예로 쓰인다.

Automaton evaluation

Automaton A 와 string x 가 주어졌을 때

$$\delta^*(q_0, x) \in F?$$

를 판정한다. Automaton이 input의 일부로 주어지는 uniform simulation problem은 단순히 하나의 고정 regular language를 판정하는 것보다 훨씬 강한 계산 문제다.

10.2 “Regular language를 못한다”는 표현을 조심해야 하는 이유

어떤 fixed Transformer가 특정 regular language를 recognize할 수 있는 것과

모든 regular language를 하나의 architecture class가 recognize할 수 있는가

는 다르다.

예를 들어 parity 자체도 regular지만 hard-attention AC^0 upper bound 아래에서는 불가능하다. 반면 “마지막 symbol이 1인가?” 같은 regular language는 trivial한 local rule이다.

따라서 발표에서는

- “Transformer는 regular language를 못한다”가 아니라,
- “해당 fixed-resource Transformer class는 모든 regular language를 cover하지 못한다”

라고 말하는 것이 안전하다.

10.3 Turing completeness와 formal-language upper bound를 동시에 기억하는 법

다음 세 문장을 따로 기억하면 된다.

1. **Fixed-depth one-pass:** circuit class로 upper-bound될 수 있다.
2. **Depth grows with n :** tree-style parallel algorithms이 가능해진다.
3. **Autoregressive steps grow with n :** serial computation을 직접 펼칠 수 있다.

같은 Transformer block을 쓰더라도 실행 graph가 달라지면 complexity class도 달라진다.

발표에서 이렇게 말하기 “Formal-language 결과를 읽을 때 language 이름보다 quantifier를 보세요. 특정 regular language를 풀 수 있느냐, 모든 regular language를 cover하느냐, automaton 자체가 input으로 주어 지느냐가 서로 다른 문제입니다. 이걸 구분하면 parity, DFA simulation, Turing completeness 결과가 한 그림 안에 들어옵니다.”

11

Arithmetic와 algorithmic reasoning을 어떻게 해석할까?

11.1 TC^0 안에도 강한 arithmetic이 있다

Threshold circuits는 단순 boolean pattern matching보다 강하다. 정수 addition, multiplication, iterated arithmetic의 상당 부분이 uniform TC^0 에 들어간다는 고전 결과가 있다.

따라서 log-precision Transformer가 한 pass에서

weighted sums, normalization, counting-like arithmetic

을 수행하는 것 자체는 TC^0 upper bound와 전혀 모순되지 않는다.

11.2 어려움은 arithmetic operation 자체보다 sequential dependency일 수 있다

예를 들어 n 개의 수를 더하는 것은 highly parallelizable하다. 반면 state가 매 step 바뀌며 다음 operation이 이전 결과에 의존하는 algorithm은

$$s_{t+1} = F(s_t, x_t)$$

꼴의 serial dependency를 가진다.

Transformer가 algorithmic reasoning에서 어려움을 겪는 이유를 단순히 “산수를 못해서”라고 보는 것보다

필요한 adaptive computational depth가 충분한가?

로 보는 관점이 더 이론과 잘 맞는다.

11.3 CoT가 제공하는 것은 externalized recurrence

Autoregressive decoder에서는

$$y_{t+1} = f(x, y_{1:t})$$

이므로 prefix가 recurrent state 역할을 한다.

일반 RNN의

$$h_{t+1} = G(h_t, x_t)$$

와 비교하면, Transformer는 state를 fixed-size hidden vector에 압축하지 않고

$$(y_1, \dots, y_t)$$

라는 growing external memory에 저장한다.

Attention은 이 history에서 필요한 record를 random-access에 가깝게 retrieve하고, FFN은 local transition을 계산한다.

보충 유도 — 논문 밖의 독자용 전개 이 관점에서 CoT의 계산력 증가는 “문장을 자연어로 설명해서”가 아니라 다음 구조 때문이다.

write token $\rightarrow$ append memory $\rightarrow$ next forward pass
 $\rightarrow$ read old memory $\rightarrow$ write next token.

즉 external-memory recurrent machine이 만들어진단.

11.4 그러면 내부 hidden reasoning은 의미가 없는가?

그렇지 않다. 한 forward pass 안에서도 각 layer는 L 단계의 computation을 한다. 다만 L 이 input length와 무관한 상수라면 asymptotic circuit depth는 고정된다.

따라서 실제 reasoning capacity를 크게 세 축으로 나눌 수 있다.

within-pass depth L , between-pass steps $t(n)$, memory/precision $p(n)$.

발표에서 이렇게 말하기 “TC⁰는 arithmetic을 못하는 class가 아닙니다. 오히려 병렬 arithmetic은 상당히 잘합니다. 문제는 이전 계산 결과를 다음 step에서 다시 써야 하는 긴 dependency chain입니다. CoT는 output prefix를 external state로 만들어 이 sequential depth를 모델 밖으로 늘려 줍니다.”

12

결과를 과장하지 않기 위한 수학적 체크리스트

12.1 Turing complete $\neq$ efficient

Turing completeness는 존재성 결과다. 어떤 computation을 simulate할 수 있다는 것과

time, space, precision, learnability

가 실용적이라는 것은 별개다.

특히 arbitrary-precision rational을 사용하는 proof는 finite-precision GPU implementation과 직접 동일시하면 안 된다.

12.2 TC^0 upper bound $\neq$ empirical impossibility

Asymptotic exact recognition theorem은

$$\forall n, \forall x \in \Sigma^n$$

에 대한 statement다.

실제 benchmark는 finite n , finite distribution, approximate accuracy를 본다. 따라서 complexity lower bound가 있어도 training distribution 안에서는 shortcut/heuristic으로 높은 accuracy가 가능하다.

12.3 CoT power $\neq$ faithful reasoning

Computational expressivity theorem은 generated token이 인간이 해석하는 reasoning trace와 일치한다는 것을 보장하지 않는다. 중간 token은 계산 resource일 수 있지만, 그 자연어 내용의 faithful explanation 여부는 별도 interpretability 문제다.

12.4 Architecture assumption을 반드시 붙인다

다음 문장들은 각각 qualifier가 필요하다.

짧게 말하면 위험한 문장	정확한 버전
Transformer는 TC^0 다.	log-precision, fixed-depth one-pass Transformer classifier에 대한 upper bound다.
Transformer는 Turing complete다.	arbitrary precision, positional encoding, hard-attention, unbounded decoding을 허용한 Pérez et al. 모델에서 성립한다.
CoT가 있으면 P를 푼다.	polynomially many decoding steps와 특정 architectural construction 아래 recognition class가 P와 일치한다.
Transformer는 regular language를 못한다.	특정 fixed-resource classes는 모든 regular language를 cover하지 못한다.
Global attention이면 sequential task도 된다.	global communication과 computational depth는 별개다.

12.5 세미나에서 가장 중요한 conceptual distinction

Expressivity : $\exists \theta$ 가 target function을 표현하는가?

Computational complexity : input length에 따라 필요한 depth/time/space는?

Learnability : optimization이 그 θ 를 찾는가?

Generalization : 더 긴 입력이나 새 distribution에도 유지되는가?

Week 4의 UAT는 첫 번째 질문에 가까웠고, 이번 주는 두 번째 질문을 다룬다.

발표에서 이렇게 말하기 “이번 주 theorem은 실제 GPT가 어떤 문제를 맞히는지 직접 예측하는 실험 법칙이 아닙니다. 무엇을 resource로 세느냐에 따라 architecture가 어느 complexity class까지 갈 수 있는지를 분류하는 결과입니다. 그래서 theorem을 인용할 때 attention, precision, depth, decoding steps를 같이 말하는 게 가장 중요합니다.”

13

발표 준비용 압축 정리

13.1 칠판에 반드시 남길 열 개 식

1. DFA recurrence

$$q_i = \delta(q_{i-1}, x_i).$$

2. Circuit hierarchy

$$\text{AC}^0 \subsetneq \text{TC}^0 \subseteq \text{NC}^1 \subseteq \text{L} \subseteq \text{NL} \subseteq \text{P}.$$

3. Log-precision Transformer upper bound

$$\text{fixed depth} + O(\log n)\text{-bit precision} \subseteq \text{uniform TC}^0.$$

4. TM transition

$$\delta(q_t, \tau_t(h_t)) = (q_{t+1}, s_t^{\text{write}}, d_t).$$

5. Pérez positional codes

$$\text{pos}(n) \text{ uses } n, \frac{1}{n}, \frac{1}{n^2}.$$

6. CoT iteration

$$y_{t+1} = f(x, y_{1:t}).$$

7. CoT class

$$\text{TIME}(t(n)) \subseteq \text{CoT}(t(n)) \subseteq \text{SPACE}(t(n) + \log n).$$

8. Layer-norm hash

$$\phi_x = \frac{\langle x, 1, -x, -1 \rangle}{\sqrt{2x^2 + 2}}.$$

9. Equality retrieval

$$\langle \phi_q, \phi_k \rangle = 1 \iff q = k.$$

10. Polynomial scratchpad

$$\text{CoT}(\text{poly}(n)) = \text{P} \quad (\text{paper의 construction assumptions 아래}).$$

13.2 발표 순서 추천

1. Week 4와 연결해 expressivity와 computational complexity를 분리한다.
2. formal language와 AC^0/TC^0 를 최소한으로 정의한다.
3. hard-attention / Hahn 결과로 fixed-depth limitation intuition을 준다.
4. Merrill-Sabharwal의 log-precision TC^0 theorem을 설명한다.
5. Pérez Turing-complete theorem을 보여 주고 일부러 “모순처럼” 제시한다.

6. precision과 decoding time이 다르므로 모순이 아님을 표로 해소한다.
7. CoT를 serial forward-pass budget으로 정의한다.
8. layer-norm hash를 직접 유도한다.
9. $O(1)$, $O(\log n)$, $O(n)$, $\text{poly}(n)$ hierarchy를 설명한다.
10. 마지막에 learnability/generalization과 분리한다.

13.3 자주 나오는 질문

Q1. Transformer가 Turing complete라면 왜 parity 같은 단순 문제에 limitation theorem이 있나?

서로 다른 모델이다. Turing-complete proof는 arbitrary precision과 unbounded decoder steps를 쓴다. Fixed-depth/limited-precision language recognizer theorem은 그런 resource를 제한한다.

Q2. TC^0 면 덧셈이나 곱셈도 못하는 약한 class인가?

아니다. Threshold circuits는 강한 parallel arithmetic을 포함한다. 한계는 주로 긴 serial dependency를 요구하는 computation에서 나타난다.

Q3. CoT가 길수록 항상 실제 accuracy가 좋아진다는 theorem인가?

아니다. 더 긴 CoT가 더 큰 computation class를 표현할 수 있게 한다는 결과다. Optimization, error accumulation, faithfulness는 별개다.

Q4. Linear CoT가 모든 regular language를 푼다면 one-pass Transformer는 regular language를 하나도 못 푸나?

아니다. 쉬운 regular language는 one-pass에서도 가능하다. Linear CoT result는 모든 regular language에 대한 coverage를 준다.

Q5. Turing-complete proof에 positional encoding이 왜 필요한가?

Unbounded computation history에서 exact time/address 정보를 구분해야 하기 때문이다. Pérez construction은 rational positional codes로 이것을 구현한다.

Q6. Layer-norm이 계산력을 늘리는 게 이상하지 않나?

일반적 의미로 layer-norm이 항상 계산력을 늘린다는 주장이 아니다. Merrill-Sabharwal construction에서는 scale invariance

$$\phi(x/i, 1/i) = \phi(x, 1)$$

를 이용해 finite precision에서 address equality를 구현하는 specific gadget로 사용한다.

Q7. Automaton은 finite memory인데 왜 one-pass Transformer가 어려워할 수 있나?

Memory size와 parallel computational depth가 다르기 때문이다. State는 작아도 n 개의 transition composition을 계산해야 한다.

13.4 혼한 수학적 실수 체크리스트

- arbitrary precision theorem과 log-precision theorem을 같은 Transformer model로 취급하지 않았는가?
- one forward pass와 autoregressive $t(n)$ passes를 구분했는가?
- AC^0 와 TC^0 를 혼동하지 않았는가?
- $\text{TC}^0 \neq \text{NC}^1$ 같은 separation을 proven fact처럼 말하지 않았는가?

- “모든 regular languages”와 “어떤 regular language”를 구분했는가?
- Turing completeness를 efficiency/learnability로 해석하지 않았는가?
- $\text{CoT}(\text{poly}(n)) = P$ 의 architectural assumptions를 생략하지 않았는가?
- layer-norm hash의 equality condition에서 Cauchy–Schwarz equality를 정확히 썼는가?
- $O(\log n)$ precision과 $O(1)$ fixed precision을 섞지 않았는가?
- formal-language exact recognition을 finite-distribution accuracy와 동일시하지 않았는가?

13.5 30초 결론

Fixed-depth Transformer 한 번의 forward pass는 매우 parallel한 계산이다. Log-precision setting에서는 그 계산을 uniform TC^0 threshold circuit로 simulate할 수 있어 긴 sequential computation에 asymptotic limitation이 생긴다. 반면 Pérez et al.의 arbitrary-precision hard-attention Transformer는 autoregressive history를 memory로 사용하면서 Turing machine의 step을 계속 simulate할 수 있어 Turing complete하다. 둘은 precision과 sequential-time assumptions가 다르므로 모순이 아니다. Merrill–Sabharwal은 이 사이를 chain-of-thought length로 정량화해, intermediate generation이 $t(n)$ 번의 serial forward pass를 제공하며 linear steps에서 모든 regular language, polynomial steps에서 적절한 architecture 가정 아래 정확히 P 의 계산력에 도달함을 보였다.

발표에서 이렇게 말하기 “한 문장으로 정리하면, Transformer의 핵심 tradeoff는 parallelism과 sequential compute입니다. 한 번의 고정-depth pass는 매우 parallel해서 TC^0 로 제한될 수 있지만, output token을 다시 input으로 넣는 순간 token 수만큼 계산 depth를 외부로 늘릴 수 있습니다. CoT가 computational resource라는 말은 정확히 이 뜻입니다.”

참고문헌 및 원문에서 확인할 위치

Jorge Pérez, Pablo Barceló, Javier Marinkovic. “Attention is Turing Complete.” *Journal of Machine Learning Research* 22, 2021.

- Sec. 2–3: Turing completeness definition, Transformer formalization, proportion invariance.
- Theorem 6: positional encodings을 가진 Transformer의 Turing completeness.
- Theorem 6 statement: positional code의 nonconstant terms $n, 1/n, 1/n^2$, one encoder + three decoder layers.
- Sec. 4: Turing-machine simulation construction.
- Sec. 5: hard attention과 arbitrary precision이 proof에서 수행하는 역할; fixed precision limitation.

<https://jmlr.org/papers/v22/20-302.html>

William Merrill, Ashish Sabharwal. “The Parallelism Tradeoff: Limitations of Log-Precision Transformers.” *TACL* 11, 2023.

- Main theorem: $O(\log n)$ precision Transformer를 logspace-uniform constant-depth threshold circuit로 simulate.
- Sec. 2: TC^0 upper bound의 computational interpretation과 parallelism tradeoff.
- Proof sections: float/arithmetic primitives와 Transformer block의 threshold-circuit simulation.

<https://aclanthology.org/2023.tacl-1.31/>

William Merrill, Ashish Sabharwal. “The Expressive Power of Transformers with Chain of Thought.” *ICLR* 2024.

- Eq. (1): $TIME(t(n)) \subseteq CoT(t(n)) \subseteq SPACE(t(n) + \log n)$ 와 time upper bound.
- Theorem 1: $|x| + 1$ decoding steps로 모든 regular language recognition.
- Theorem 2: one decoding step per Turing-machine step simulation.
- Lemma 1–2: layer-norm hash와 equality check.
- Theorem 3–4: time/space upper bounds.
- Sec. 4–5: logarithmic, linear, polynomial CoT의 complexity interpretation.

https://proceedings.iclr.cc/paper_files/paper/2024/hash/1f59721c106ea80f613299039112f651-Abstract-Conference.html

Michael Hahn. “Theoretical Limitations of Self-Attention in Neural Sequence Models.” *TACL* 8, 2020.

- Parity와 Dyck를 이용한 fixed-resource self-attention limitation.
- Input length에 따라 layer/head resource를 키우지 않을 때의 formal-language limitations.

<https://aclanthology.org/2020.tacl-1.11/>

Yiding Hao, Dana Angluin, Robert Frank. “Formal Language Recognition by Hard Attention Transformers: Perspectives from Circuit Complexity.” TACL 10, 2022.

- UHAT/GUHAT의 AC^0 upper bound.
- AHAT가 MAJORITY와 DYCK-1을 recognize할 수 있는 separation.

<https://aclanthology.org/2022.tacl-1.46/>

Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, Cyril Zhang. “Transformers Learn Shortcuts to Automata.” ICLR 2023.

- 모든 finite automaton에 대한 polynomial-size $O(\log n)$ -depth shortcut construction.
- algebraic structure에 따른 shallower construction과 empirical shortcut learning.
- length generalization에서 shortcut brittleness에 대한 실험적 분석.

<https://openreview.net/forum?id=De4FYqjFueZ>

주의— 원문과 이 노트의 경계 Circuit-class 정의, DFA transition을 balanced composition으로 쓰는 $O(\log n)$ 직관, Pérez construction을 “event log + latest write lookup”으로 설명한 부분, CoT를 externalized recurrence로 해석한 부분은 원 theorem을 이해하기 위한 보충 전개다. Pérez et al.과 Merrill–Sabharwal의 full simulation proof는 많은 coordinate-level gadget을 포함하므로 이 노트는 theorem statement와 proof mechanism을 보존하되 모든 coordinate construction을 복제하지 않았다. 또한 $TC^0 \neq NC^1$, $L \neq P$ 같은 separation은 표준 conjecture이지 증명된 사실로 다루지 않았다.